Multi-Agent Synchronization of Twin Mobile Robots

A Low-Cost ESP-NOW–Based Leader–Follower Architecture with Encoder Odometry and Closed-Loop PID Trajectory Tracking

Undergraduate Control Systems Engineering Evaluation

Department of Electrical, Electronic and Mechatronic Engineering

July 29, 2026

Abstract

This report documents the design, mathematical modeling, and experimental evaluation of a low-cost multi-agent robotic testbed comprising two differential-drive mobile robots operating under a leader–follower synchronization scheme. Each robot is instrumented with an ESP8266 system-on-chip, quadrature wheel encoders, and an H-bridge motor driver, and the two agents communicate over Espressif's connectionless ESP-NOW protocol rather than a conventional Wi-Fi/TCP-IP stack, thereby minimizing latency and eliminating the need for a shared access point. A discrete-time unicycle kinematic model is derived and combined with encoder-based odometry to provide each agent with a real-time pose estimate. A cascaded proportional–integral–derivative (PID) control law is used at the follower agent to track a time-varying reference trajectory broadcast by the leader, with an outer trajectory-tracking loop and inner per-wheel velocity loops. Beyond the baseline synchronization task, three technical enhancements are proposed and analyzed: dynamic formation reconfiguration through parametric offset injection, packet-loss-resilient dead reckoning using encoder-only fallback during communication dropouts, and adaptive gain scheduling as a function of tracking error magnitude and estimated relative velocity. The report presents the complete system architecture, the governing equations of motion, controller derivations with stability considerations rooted in Lyapunov-style reasoning, a phased implementation and testing methodology, and an analysis framework for latency, tracking-error convergence, and scalability toward N-agent swarms. The work demonstrates that meaningful multi-agent consensus behavior, historically associated with expensive robotic platforms, can be reproduced on consumer-grade microcontroller hardware costing under twenty United States dollars per agent, making the platform suitable for classroom instruction in distributed control and networked robotics.

Keywords: multi-agent systems, leader–follower control, ESP-NOW, PID control, differential-drive kinematics, dead reckoning, distributed consensus, low-cost robotics

Contents

1 Abstract and Introduction **4**

1.1 Background: Multi-Agent Systems and Distributed Consensus 4

1.2 Motivation for Low-Cost Hardware 4

1.3 Report Organization 5

2 System Architecture and Hardware Design **5**

2.1 Overview 5

2.1.1 Bill of Materials (per agent) 6

2.2 Compute and Communication: ESP8266 7

2.2.1 Why ESP-NOW rather than Wi-Fi/TCP-IP 7

2.2.2 Network Topology 8

2.3 Actuation: Motor Driver Stage 8

2.4 Sensing: Quadrature Wheel Encoders 8

2.5 Power Distribution 9

3 Mathematical Modeling and Control Algorithm **9**

3.1 Differential-Drive (Unicycle) Kinematics 9

3.2 Discrete-Time Encoder Odometry 10

3.3 Leader–Follower Tracking Error Coordinates 10

3.4 PID Control Law: Cascaded Architecture 11

3.5 Continuous-Time Stability Considerations 12

4 Novel Technical Enhancements **12**

4.1 Dynamic Formation Reconfiguration 13

4.2 Packet-Loss-Resilient Dead Reckoning 13

4.3 Adaptive Gain Scheduling 14

5 Implementation and Testing Methodology **15**

5.1 Phase 1: Hardware Assembly and Bring-Up 15

5.2 Phase 2: Sensor Validation and Odometry Calibration 15

5.3 Phase 3: Wireless Communication Validation 15

5.4 Phase 4: Closed-Loop Controller Tuning 16
5.5 Phase 5: Integrated Leader–Follower Trials and Performance Logging 16

6 Expected Results, Performance Analysis, and Conclusion 16

6.1 Communication Latency 16
6.2 Tracking Error Convergence 17
6.3 Resilience to Communication Dropout 17
6.4 Summary of Contributions 17
6.5 Scalability to N-Agent Swarms 18

1 Abstract and Introduction

1.1 Background: Multi-Agent Systems and Distributed Consensus

A multi-agent system (MAS) consists of a collection of autonomous, spatially distributed agents that must coordinate their actions to achieve a global objective that no single agent can accomplish, or accomplish as efficiently, in isolation. In robotics, multi-agent systems underpin applications ranging from warehouse logistics fleets and agricultural swarms to search-and-rescue teams and formation-flying satellites. The central theoretical challenge in all such systems is *distributed consensus*: how a set of agents, each possessing only local and partial information about the state of the collective, can converge on a shared quantity of interest — position, heading, velocity, or a more abstract decision variable — using only peer-to-peer communication rather than a centralized coordinator.

Formally, consensus problems are often posed on a communication graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where each vertex $v_i \in \mathcal{V}$ represents an agent and each edge $(i, j) \in \mathcal{E}$ represents an available communication link. Each agent updates its own state x_i according to a local rule of the form

$$\dot{x}_i(t) = \sum_{j \in \mathcal{N}_i} a_{ij} \left(x_j(t) - x_i(t) \right) \tag{1}$$

where $\mathcal{N}_i$ is the neighbor set of agent i and $a_{ij} > 0$ are coupling weights. Under mild connectivity assumptions on $\mathcal{G}$, this class of protocols is known to drive all agent states toward a common value. The present project is a deliberately simplified, hardware-realizable instance of this broader theory: two agents ($|\mathcal{V}| = 2$), a single directed communication edge (leader $\rightarrow$ follower), and a consensus variable that is not an abstract scalar but a full six-parameter kinematic pose (x, y, θ) together with its time derivatives. This reduction to the two-agent case is intentional: it isolates the core synchronization primitives — state broadcast, latency compensation, and closed-loop tracking — from the additional complexity of graph topology and network scheduling that arises once $N > 2$, while still preserving a direct and well-understood extension path to larger swarms (discussed in Section 6).

1.2 Motivation for Low-Cost Hardware

Multi-agent robotics research has traditionally relied on platforms such as the Khepera, e-puck, or Crazyflie families, or on custom robots built around Raspberry Pi or NVIDIA Jetson compute modules with dedicated radio modules (XBee, nRF24L01, or full 802.11 Wi-Fi with a central router). These platforms are pedagogically excellent but carry per-unit costs that are frequently prohibitive for undergraduate laboratories seeking to give every student, or every small team, hands-on access to a genuine multi-robot system rather than a simulation.

The ESP8266 microcontroller, originally developed as a low-cost Wi-Fi transceiver for embedded internet-of-things applications, presents an attractive alternative. It integrates a 32-bit Tensilica L106 processor clocked at 80–160 MHz, an 802.11 b/g/n radio, and sufficient general-purpose

input/output (GPIO) to drive a motor controller and read encoder signals, all on a module that retails for a few dollars. Critically for this project, Espressif exposes a connectionless communication protocol, **ESP-NOW**, that operates beneath the standard Wi-Fi association layer: two ESP8266 devices can exchange short data frames directly, addressed by MAC address, without joining a common access point, negotiating an IP address, or maintaining a TCP session. This yields communication latencies on the order of a few milliseconds and removes an entire class of infrastructure dependency (routers, DHCP servers, subnet configuration) that would otherwise complicate a teaching laboratory deployment.

The motivation for this project is therefore twofold:

(i) To demonstrate that the theoretical machinery of multi-agent consensus and leader–follower control — normally taught with block diagrams and simulated in MATLAB/Simulink — can be instantiated on physical hardware costing less than the price of a textbook per agent, closing the gap between abstract control theory and embodied, real-time behavior.

(ii) To provide a complete, self-contained reference design — covering hardware selection, wireless network architecture, kinematic modeling, controller synthesis, and experimental validation — that can serve as a template for subsequent capstone projects extending the two-agent case to N-agent swarm coordination.

1.3 Report Organization

Section 2 details the electromechanical architecture of each robot and the ESP-NOW network topology binding the two agents. Section 3 derives the unicycle kinematic model, the encoder-based odometry update equations, and the PID control law used for trajectory tracking, together with a discussion of stability. Section 4 presents three enhancements beyond the baseline synchronization task: dynamic formation reconfiguration, packet-loss-resilient dead reckoning, and adaptive gain scheduling. Section 5 describes the phased implementation and testing methodology. Section 6 discusses expected results, analyzes performance in terms of latency and tracking error, and concludes with a discussion of scalability to N-agent swarms.

2 System Architecture and Hardware Design

2.1 Overview

Each mobile agent is a two-wheeled differential-drive robot built around a common electromechanical stack. The two agents are electrically and mechanically identical, which simplifies both the control derivation (a single kinematic and dynamic model applies to both) and the bill of materials. The only asymmetry between the two units is introduced in firmware: one unit is configured, at compile time, to run the *leader* role, and the other the *follower* role.

2.1.1 Bill of Materials (per agent)

Table 1: Hardware components per mobile agent.

Component	Function	Typical Spec.
ESP8266 (NodeMCU / Wemos D1 Mini)	Compute, wireless communication, PWM generation	80/160 MHz, 4 MB flash, 802.11 b/g/n
L298N / TB6612FNG motor driver	H-bridge current amplification for DC gear-motors	2-channel, PWM enable
DC gear-motor with quadrature encoder (×2)	Actuation and incremental position/velocity sensing	≈200–400 counts/rev at wheel shaft
Differential-drive chassis	Mechanical platform, wheel/caster support	Two driven wheels, one caster
Li-ion battery pack (2S, ∼7.4 V) with buck regulator	Power source for logic and motors	7.4 V nominal, regulated 5 V/3.3 V rails
Optional 9-DoF IMU (MPU-6050)	Heading,	3-axis gyro + accelerometer

2.2 Compute and Communication: ESP8266

The ESP8266 is selected as the sole onboard compute element for each agent. It performs three concurrent functions within a single super-loop / timer-interrupt firmware architecture:

1. **Encoder pulse counting**, serviced by GPIO interrupt service routines (ISRs) attached to the encoder channel-A pins of both wheels, incrementing signed pulse counters that are periodically read and reset by the main control loop.
2. **Control-law execution**, running the PID computation described in Section 3 at a fixed sample period T_s (nominally $T_s = 20\,\text{ms}$, i.e., a $50\,\text{Hz}$ control rate), producing left- and right-wheel PWM duty-cycle commands.
3. **ESP-NOW packet transmission and reception**, broadcasting the agent's own pose/velocity state and, on the follower, ingesting the leader's most recently received state for use as the tracking reference.

2.2.1 Why ESP-NOW rather than Wi-Fi/TCP-IP

Conventional Wi-Fi communication between two microcontrollers requires both devices to associate with an access point, obtain IP addresses (typically via DHCP), and open a transport-layer connection (TCP or UDP socket) before any application data can be exchanged. Each of these steps adds both latency and a dependency on infrastructure that may not be reliably present in a teaching-laboratory or field setting. ESP-NOW instead operates directly on top of the 802.11 physical and MAC layers: a device registers its peer's 48-bit MAC address, and thereafter transmits short (up to 250-byte) application payloads directly, connectionless and without IP addressing. Because no association, handshake, or routing is required, single-hop ESP-NOW transmissions typically complete in low single-digit milliseconds, which is small relative to the $T_s = 20\,\text{ms}$ control period and therefore does not dominate the closed-loop latency budget.

A further advantage relevant to the leader–follower architecture is that ESP-NOW natively supports broadcast delivery: the leader can transmit its state once per cycle to a broadcast MAC address, and it is received by the follower (and, if the system were extended, by any number of additional followers) without the leader needing to manage per-peer acknowledgment or unicast overhead individually — a property directly relevant to the N-agent scalability discussion in Section 6.

2.2.2 Network Topology

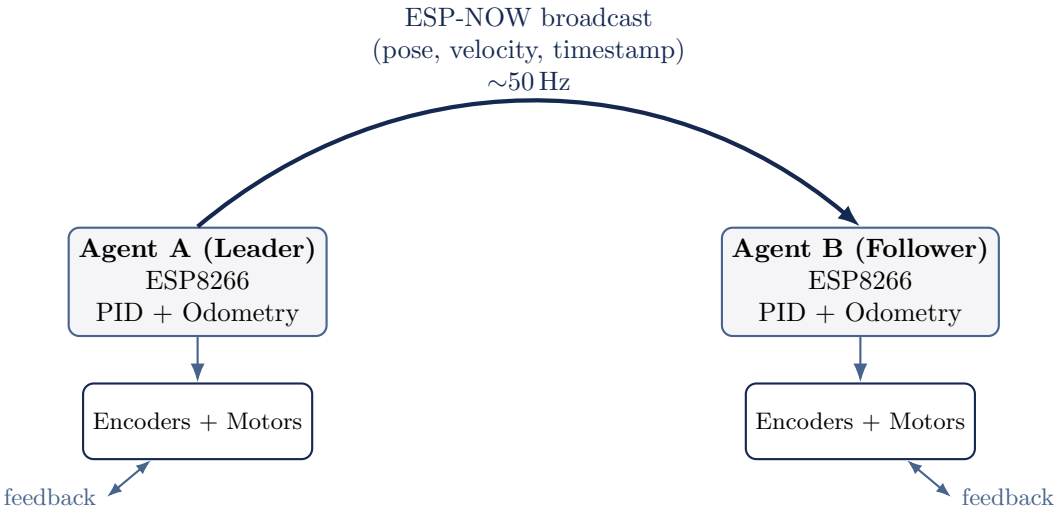


Figure 1: ESP-NOW network topology: a single directed broadcast link from leader to follower, with each agent independently closing its own low-level actuation loop against its encoders.

The leader periodically broadcasts a compact state packet containing its estimated pose $(\hat{x}_L, \hat{y}_L, \hat{\theta}_L)$, its commanded/estimated linear and angular velocity (v_L, ω_L), a monotonically increasing sequence number, and a microcontroller-clock timestamp. The follower's radio ISR stores the most recently received packet into a double-buffered structure so that the main control loop always operates on a self-consistent snapshot of the leader's state, avoiding torn reads between the ISR and the control task.

2.3 Actuation: Motor Driver Stage

Each driven wheel is actuated by a brushed DC gear-motor through a dual H-bridge driver (L298N or the more efficient TB6612FNG). The ESP8266 generates two PWM channels (one per wheel) at a switching frequency of approximately 1 kHz–5 kHz, together with direction-control logic-level outputs. The duty cycle $d \in [-1, 1]$ produced by the control law of Section 3 is mapped to a PWM register value and a forward/reverse GPIO state:

$$\mathrm{PWM}_{\mathrm{reg}} = \lfloor |d| \cdot \mathrm{PWM}_{\max} \rfloor, \qquad \mathrm{DIR} = \mathrm{sign}(d). \tag{2}$$

A dead-zone compensation term is added in firmware to counter the static friction (stiction) of the gear-motors, since small commanded duty cycles otherwise fail to produce any shaft rotation.

2.4 Sensing: Quadrature Wheel Encoders

Each wheel is fitted with a magnetic or optical quadrature encoder producing two phase-shifted pulse trains (channels A and B), from which both the magnitude and direction of rotation can

be recovered. Channel A is attached to an ESP8266 hardware interrupt pin; on each rising edge, the ISR samples channel B to determine direction and increments or decrements a signed count. The wheel's incremental rotation over a control period T_s is

$$\Delta\phi = \frac{2\pi\,\Delta N}{\mathrm{CPR}} \tag{3}$$

where ΔN is the net encoder count accumulated during the interval and CPR is the number of counts per wheel revolution (accounting for gear reduction).

2.5 Power Distribution

Motor and logic supplies are separated to prevent PWM switching transients on the motor rail from corrupting the 3.3 V logic supply that powers the ESP8266 and its radio front-end. A 2-cell lithium-ion pack (nominal 7.4 V) feeds the H-bridge directly, while a switching buck regulator steps this down to a clean 5 V rail for the encoder pull-ups and a linear or switching 3.3 V rail for the microcontroller. A common ground reference is maintained across both domains. A decoupling capacitor bank ($\geq 100\,\mu$F bulk plus $0.1\,\mu$F ceramic) is placed close to the ESP8266's 3.3 V pin, since the module's radio transmit bursts draw current spikes of several hundred milliamps that would otherwise cause brown-out resets.

Table 2: Power domain summary.

Rail	Nominal Voltage	Consumers	Notes
Battery (raw)	7.4 V (2S Li-ion)	Motor driver VIN	Direct, unregulated
5 V regulated	5.0 V	Encoder pull-ups, logic-level shifters	Buck converter
3.3 V regulated	3.3 V	ESP8266 core, radio	Low-noise regulator, decoupled

3 Mathematical Modeling and Control Algorithm

3.1 Differential-Drive (Unicycle) Kinematics

Each agent is modeled as a rigid body constrained to planar motion with two independently driven wheels separated by track width L and wheel radius r. Let the pose of an agent at time t be $\mathbf{q}(t) = \begin{bmatrix} x(t) & y(t) & \theta(t) \end{bmatrix}^T$, where (x, y) is the position of the point midway between the two wheels in a fixed world frame and θ is the heading angle measured from the world x-axis. Denoting the angular velocities of the left and right wheels as ω_l and ω_r, the wheel linear speeds are $v_l = r\,\omega_l$ and $v_r = r\,\omega_r$. The standard unicycle kinematic model follows from a rigid-body, no-slip (nonholonomic) assumption:

$$v = \frac{v_r + v_l}{2}, \qquad \omega = \frac{v_r - v_l}{L} \tag{4}$$

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega \tag{5}$$

which may be written compactly as

$$\dot{\mathbf{q}} = \begin{bmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} = G(\mathbf{q})\,\mathbf{u} \tag{6}$$

with $\mathbf{u} = [v, \omega]^T$ the control input vector. This is the canonical nonholonomic unicycle model: the robot cannot instantaneously move sideways, which is the same structural constraint present in car-like and differential-drive vehicles generally, and it is this constraint that motivates the specific tracking-error coordinate transformation used in Section 3.3.

3.2 Discrete-Time Encoder Odometry

Because the onboard control loop executes at discrete sample instants $t_k = kT_s$, the continuous kinematic model is realized in firmware via forward-Euler integration driven directly by encoder counts, avoiding any dependency on a commanded (rather than actually achieved) velocity.

At each sample k, let $\Delta N_l[k]$ and $\Delta N_r[k]$ be the net encoder counts accumulated on the left and right wheels since the previous sample. The incremental wheel travel distances are

$$\Delta s_l[k] = \frac{2\pi r}{\mathrm{CPR}}\,\Delta N_l[k], \qquad \Delta s_r[k] = \frac{2\pi r}{\mathrm{CPR}}\,\Delta N_r[k] \tag{7}$$

from which the incremental linear and angular displacement of the robot's centroid are

$$\Delta s[k] = \frac{\Delta s_r[k] + \Delta s_l[k]}{2}, \qquad \Delta\theta[k] = \frac{\Delta s_r[k] - \Delta s_l[k]}{L}. \tag{8}$$

The pose estimate is then updated using a midpoint (or "exact" arc) integration rule, which is preferred over a naive forward-Euler update because it materially reduces heading-dependent bias over many iterations:

$$\hat{\theta}[k] = \hat{\theta}[k-1] + \Delta\theta[k] \tag{9}$$

$$\hat{x}[k] = \hat{x}[k-1] + \Delta s[k]\cos\Big(\hat{\theta}[k-1] + \tfrac{\Delta\theta[k]}{2}\Big) \tag{10}$$

$$\hat{y}[k] = \hat{y}[k-1] + \Delta s[k]\sin\Big(\hat{\theta}[k-1] + \tfrac{\Delta\theta[k]}{2}\Big) \tag{11}$$

This recursive scheme constitutes the odometry-based localization that each agent runs entirely locally, requiring no external positioning infrastructure and forming the state that the leader broadcasts and the follower uses both as its own feedback and as the basis for computing tracking error relative to the leader.

3.3 Leader–Follower Tracking Error Coordinates

For a follower with pose $\mathbf{q}_F = [x_F, y_F, \theta_F]^T$ tracking a (possibly time-varying, offset) reference pose $\mathbf{q}_R = [x_R, y_R, \theta_R]^T$ derived from the leader's broadcast state, the global position error is transformed into the follower's body frame to obtain a control-relevant error vector $\mathbf{e} =$

$[e_x, e_y, e_\theta]^T$:

$$\begin{bmatrix} e_x \\ e_y \\ e_\theta \end{bmatrix} = \begin{bmatrix} \cos\theta_F & \sin\theta_F & 0 \\ -\sin\theta_F & \cos\theta_F & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_R - x_F \\ y_R - y_F \\ \theta_R - \theta_F \end{bmatrix}. \tag{12}$$

This is the standard body-frame tracking-error transformation used throughout mobile-robot trajectory-tracking literature; e_x is the along-heading position error, e_y is the lateral (cross-track) error, and e_θ is the heading error. A kinematic tracking control law of the form

$$v_c = v_R \cos e_\theta + K_x e_x \tag{13}$$

$$\omega_c = \omega_R + K_y v_R \,\mathrm{sinc}(e_\theta)\, e_y + K_\theta e_\theta \tag{14}$$

(with $K_x, K_y, K_\theta > 0$ design gains and $\mathrm{sinc}(e_\theta) = \sin(e_\theta)/e_\theta$) can be shown, via the Lyapunov candidate

$$V = \tfrac{1}{2}(e_x^2 + e_y^2) + \tfrac{1-\cos e_\theta}{K_y} \tag{15}$$

to drive $\mathbf{e} \to 0$ asymptotically, provided v_R and ω_R remain bounded and $\dot{V} \le 0$ along trajectories, which follows directly by differentiating V and substituting the closed-loop error dynamics. This kinematic-level law furnishes the outer-loop reference velocities (v_c, ω_c) that are subsequently converted to individual wheel-speed setpoints and regulated by the PID loops described next; the present project layers a PID-based realization on top of this kinematic structure rather than treating the two as separate design philosophies.

3.4 PID Control Law: Cascaded Architecture

The controller implemented on the follower agent is organized as a two-level cascade:

Outer loop (trajectory tracking). Executes the kinematic law of Section 3.3 to convert pose error into desired body-frame velocity commands (v_c, ω_c).

Inner loop (per-wheel velocity regulation). Converts (v_c, ω_c) into individual wheel-speed setpoints

$$\omega_r^* = \frac{v_c + \frac{L}{2}\omega_c}{r}, \qquad \omega_l^* = \frac{v_c - \frac{L}{2}\omega_c}{r} \tag{16}$$

and regulates each wheel's actual angular velocity (obtained by differentiating the encoder count over T_s) to its setpoint using an independent discrete PID controller. For a generic wheel with instantaneous speed error $e[k] = \omega^*[k] - \hat{\omega}[k]$, the discrete PID control law implemented in firmware is

$$u[k] = K_p\, e[k] \;+\; K_i\, T_s \sum_{n=0}^{k} e[n] \;+\; K_d\, \frac{e[k] - e[k-1]}{T_s} \tag{17}$$

or, in the numerically preferred incremental (velocity) form actually used to avoid integral recomputation and to simplify anti-windup clamping,

$$\Delta u[k] = K_p\big(e[k] - e[k-1]\big) + K_i T_s\, e[k] + \frac{K_d}{T_s}\big(e[k] - 2e[k-1] + e[k-2]\big) \tag{18}$$

$$u[k] = u[k-1] + \Delta u[k], \qquad u[k] \in [-1, 1] \text{ (saturated duty cycle).} \tag{19}$$

Integral anti-windup is implemented via conditional integration: the integral accumulator is frozen whenever the unsaturated control output exceeds the actuator limits, preventing the classical windup transient that would otherwise appear as pronounced overshoot when the follower re-acquires the leader's trajectory after a large transient (e.g., following a communication dropout, see Section 4.2).

3.5 Continuous-Time Stability Considerations

Treating a single wheel-speed loop in continuous time, the DC gear-motor with driver is well approximated near its operating point by a first-order transfer function

$$G_m(s) = \frac{\Omega(s)}{U(s)} = \frac{K_m}{\tau_m s + 1} \tag{20}$$

where K_m is the motor gain (rad/s per unit duty cycle) and τ_m is the electromechanical time constant. With a PID compensator $C(s) = K_p + K_i/s + K_d s$, the closed-loop characteristic equation is

$$1 + C(s)\,G_m(s) = 0 \implies \tau_m s^3 + (1 + K_d K_m)s^2 + K_p K_m s + K_i K_m = 0. \tag{21}$$

Routh–Hurwitz analysis of this third-order polynomial yields the necessary and sufficient stability condition (for positive coefficients, which holds by construction since all gains and $K_m, \tau_m > 0$):

$$(1 + K_d K_m)(K_p K_m) > \tau_m K_i K_m, \tag{22}$$

i.e., the product of the effective damping and proportional terms must exceed the product of the plant time constant and the integral gain. This inequality provides the theoretical basis for the empirical gain-tuning procedure of Section 5: it explains why excessive integral gain relative to proportional/derivative gain destabilizes the inner velocity loop, and it is used as a sanity check on gain sets identified experimentally via Ziegler–Nichols-style tuning before deployment on hardware.

For the outer kinematic tracking loop, local exponential stability of the origin of the error dynamics $\dot{\mathbf{e}} = f(\mathbf{e}, v_R, \omega_R)$ under the control law of Eq. (14)–(15) follows from linearizing about $\mathbf{e} = 0$ under the standard persistency-of-excitation assumption that ω_R does not identically vanish, which is satisfied for any trajectory containing curvature (a straight-line, constant-velocity leader trajectory is the one degenerate case requiring special handling, discussed operationally in Section 5).

4 Novel Technical Enhancements

The baseline system described in Sections 2–3 realizes single-directional leader–follower synchronization. Three enhancements extend this baseline toward more robust and more flexible

multi-agent behavior.

4.1 Dynamic Formation Reconfiguration

Rather than commanding the follower to converge to the leader's exact pose, a constant or time-varying *formation offset* $\boldsymbol{\delta}(t) = [\delta_x(t), \delta_y(t), \delta_\theta(t)]^T$, expressed in the leader's body frame, is injected into the reference pose computation:

$$\mathbf{q}_R(t) = \mathbf{q}_L(t) + R(\theta_L(t))\,\boldsymbol{\delta}(t) \tag{23}$$

where $R(\theta_L)$ is the standard 2-D rotation matrix mapping the offset from the leader's body frame into the world frame. Setting $\boldsymbol{\delta} = \mathbf{0}$ recovers pure pose-following; setting $\delta_x < 0$ commands a trailing (convoy) formation, while a nonzero δ_y commands a side-by-side or staggered formation. Because $\boldsymbol{\delta}(t)$ is permitted to vary with time (subject to a rate limit to keep the resulting reference velocity within actuator bounds), the follower can be commanded to smoothly transition between formations — for example, from trailing to side-by-side — without discontinuities in the commanded velocity, by low-pass filtering $\boldsymbol{\delta}(t)$ before use:

$$\dot{\boldsymbol{\delta}}_{\text{filt}} = \frac{1}{\tau_\delta}\left(\boldsymbol{\delta}_{\text{cmd}} - \boldsymbol{\delta}_{\text{filt}}\right). \tag{24}$$

The offset command itself may originate from a pre-scripted formation schedule, from an operator input relayed by the leader, or — in the natural extension to $N > 2$ agents — from a distinct offset assigned to each follower index, enabling arbitrary polygonal or line formations from the same underlying control architecture.

4.2 Packet-Loss-Resilient Dead Reckoning

Because ESP-NOW, like any wireless link, is subject to occasional packet loss (interference, distance, or radio congestion), the follower cannot assume that a fresh leader-state packet is available every control cycle. A naive implementation that simply holds the last received leader pose constant during a dropout will systematically under-command the follower whenever the leader is in motion, since the reference pose used in Eq. (11) will fail to advance while the follower continues to close in on an increasingly stale target.

The proposed mitigation is a local dead-reckoning extrapolation of the leader's state during a dropout, using the leader's last-known velocity to forward-propagate its pose through the unicycle kinematics of Section 3.1:

$$\hat{\mathbf{q}}_L[k] = \begin{cases} \mathbf{q}_L^{\text{rx}}[k], & \text{if a packet was received at step } k \\ \hat{\mathbf{q}}_L[k-1] + T_s\, G(\hat{\theta}_L[k-1]) \begin{bmatrix} v_L^{\text{last}} \\ \omega_L^{\text{last}} \end{bmatrix}, & \text{otherwise} \end{cases} \tag{25}$$

where $v_L^{\text{last}}, \omega_L^{\text{last}}$ are held from the most recently received packet. A packet-age counter k_{age}

tracks how many consecutive cycles have elapsed since the last successful reception; the extrapolated velocity term is exponentially attenuated as k_{age} grows,

$$\begin{bmatrix} v_L^{\text{eff}} \\ \omega_L^{\text{eff}} \end{bmatrix} = e^{-\lambda\, k_{\text{age}} T_s} \begin{bmatrix} v_L^{\text{last}} \\ \omega_L^{\text{last}} \end{bmatrix}, \tag{26}$$

so that a prolonged dropout causes the extrapolated reference to decay smoothly toward a stationary hold rather than continuing to integrate an assumed constant velocity indefinitely, which would otherwise accumulate unbounded position error during long outages. This design directly parallels dead-reckoning practice in inertial navigation, where velocity-integrated position estimates are trusted only over short horizons before being down-weighted or re-anchored against an external reference —here, the external reference is simply the next successfully received ESP-NOW packet, which resets $k_{\text{age}} \to 0$ and snaps $\hat{\mathbf{q}}_L$ back to ground truth.

4.3 Adaptive Gain Scheduling

The fixed-gain PID design of Section 3.4 is tuned for a nominal operating regime, but a single fixed gain set is necessarily a compromise between aggressive tracking (large gains, fast convergence, but higher overshoot and sensitivity to encoder quantization noise at low error) and conservative tracking (small gains, smooth low-speed behavior, but sluggish recovery from large transients such as a formation-offset step change or the aftermath of a dead-reckoning dropout). Adaptive gain scheduling addresses this by making the effective PID gains a function of the instantaneous tracking-error magnitude $\|\mathbf{e}\|$ and the estimated relative velocity between leader and follower.

A piecewise-continuous scheduling function is used in preference to a fully continuous nonlinear gain surface, both for firmware simplicity and for predictable, individually testable behavior in each regime:

$$K_p(\|\mathbf{e}\|) = K_p^{\text{base}} \left[1 + \alpha \cdot \text{sat}\left(\frac{\|\mathbf{e}\| - e_{\text{th}}}{e_{\text{sat}}} \right) \right] \tag{27}$$

where $\text{sat}(\cdot)$ clamps its argument to $[0, 1]$, e_{th} is an error threshold below which the base gain applies unmodified, e_{sat} sets the error magnitude at which the gain boost saturates, and $\alpha > 0$ is the maximum fractional gain increase. An analogous, independently tunable reduction is applied to the derivative gain at large error to avoid derivative-kick amplification of encoder quantization noise once the error (and hence its sample-to-sample fluctuation) grows large:

$$K_d(\|\mathbf{e}\|) = K_d^{\text{base}} \left[1 - \beta \cdot \text{sat}\left(\frac{\|\mathbf{e}\| - e_{\text{th}}}{e_{\text{sat}}} \right) \right], \qquad 0 < \beta < 1. \tag{28}$$

The net effect is a controller that behaves as a conservative, smooth regulator during steady-state formation-keeping (small $\|\mathbf{e}\|$) but transiently stiffens its proportional response — while relaxing derivative sensitivity — during large-error transients such as a reconfiguration command or dropout recovery, thereby reducing settling time without permanently sacrificing steady-state smoothness. Because $K_p(\cdot)$ and $K_d(\cdot)$ are recomputed once per control cycle from already-available quantities ($\mathbf{e}[k]$), this scheme adds negligible computational overhead to the ESP8266's

control loop.

5 Implementation and Testing Methodology

The system was brought up and validated through a phased progression, with each phase gated on the successful, repeatable completion of the phase before it, rather than against a fixed calendar schedule. This approach reflects standard embedded-systems integration practice, in which timing budgets are set by successful bring-up of the layer beneath, not by a wall clock.

5.1 Phase 1: Hardware Assembly and Bring-Up

Each chassis is assembled with its motor pair, encoder pair, and caster wheel, followed by wiring of the motor driver and power distribution as specified in Section 2. Bring-up begins at the component level: the battery and regulator rails are verified independently under bench load before the ESP8266 is connected, to avoid exposing the microcontroller to an unregulated or noisy supply. Each motor channel is then exercised open-loop (fixed PWM duty cycle, no feedback) to confirm correct rotation direction and to verify that the H-bridge direction logic matches the sign convention assumed in the kinematic model; a mismatched wheel polarity at this stage would otherwise manifest later as an apparently unstable control loop rather than an obviously miswired motor.

5.2 Phase 2: Sensor Validation and Odometry Calibration

With actuation confirmed, each encoder channel is validated in isolation by manually rotating the wheel through a known number of revolutions and confirming the expected pulse count, which calibrates the effective CPR (accounting for gear-train ratio and any encoder-disc under/over-count). The full odometry pipeline of Section 3.2 is then exercised by driving the robot through simple, repeatable motion primitives — a straight-line traverse of known length, and an in-place rotation of a known angle — and comparing the resulting $(\hat{x}, \hat{y}, \hat{\theta})$ estimate against the physically measured ground truth. Systematic discrepancies at this stage are attributed to and corrected via two calibration parameters: an effective wheel-radius scale factor (correcting for tire compression and wheel wear) and an effective track-width scale factor (correcting for wheel slip during rotation), following the standard UMBmark-style systematic-odometry-error calibration approach used broadly in mobile-robotics practice.

5.3 Phase 3: Wireless Communication Validation

ESP-NOW peer registration is validated first at short range with both units stationary, confirming successful packet exchange, correct payload deserialization, and a stable sequence-number progression (used to detect packet loss). Round-trip and one-way latency are characterized by timestamping packets at transmission and again at reception (with both devices' clocks

periodically cross-synchronized via an initial handshake exchange), across a range of physical separations and with intentionally introduced radio-frequency interference (e.g., a co-located Wi-Fi access point operating on an overlapping channel) to characterize the packet-loss-versus-range and packet-loss-versus-interference behavior that the dead-reckoning logic of Section 4.2 must tolerate.

5.4 Phase 4: Closed-Loop Controller Tuning

Inner-loop wheel-speed PID gains are tuned first, with the robot's driven wheels lifted clear of the ground so that pure wheel-speed regulation can be validated without confounding whole-body kinematic effects. A relay-feedback or manual step-response procedure is used to identify approximate motor gain K_m and time constant τ_m, from which an initial Ziegler–Nichols gain set is computed and then refined by observing step-response overshoot and settling time on the bench. Only once each wheel's inner loop is well-behaved in isolation is the robot placed on the ground and the outer kinematic tracking loop enabled, first against a stationary reference pose (pure point-stabilization), then against a slowly moving reference, and finally against the full time-varying leader trajectory broadcast over ESP-NOW.

5.5 Phase 5: Integrated Leader–Follower Trials and Performance Logging

With both inner and outer loops validated independently, the two agents are operated together under a sequence of leader trajectories of increasing complexity: a straight-line traverse, a constant-curvature arc, and a trajectory combining straight segments, turns, and velocity changes intended to exercise the follower's transient response. Throughout each trial, both agents log their own odometry-derived pose estimate, the received/extrapolated reference pose, the instantaneous tracking-error vector $\mathbf{e}[k]$, and the ESP-NOW packet sequence numbers and reception timestamps, either to onboard flash for later retrieval or streamed over a debug serial/Wi-Fi link to a host computer for real-time visualization. This logged dataset forms the basis of the performance analysis in Section 6, and the same logging infrastructure is reused, without modification, to evaluate the enhancements of Section 4 by deliberately invoking formation-offset changes, artificially inducing communication dropouts, and comparing tracking error with adaptive gain scheduling enabled versus disabled.

6 Expected Results, Performance Analysis, and Conclusion

6.1 Communication Latency

Given ESP-NOW's connectionless, single-hop design, one-way transmission latency is expected to remain in the low single-digit-millisecond range under nominal (uncongested, short-range) conditions, substantially smaller than the $T_s = 20\,\text{ms}$ control period. Under this condition, the latency's effect on tracking performance is expected to be dominated not by the raw transmission delay itself but by the *packet loss rate*, since an occasional dropped packet (handled by the

dead-reckoning scheme of Section 4.2) is expected to produce a materially larger transient error excursion than the sub-period latency of a successfully delivered packet. Latency and loss rate are both expected to degrade gracefully with increasing range and increasing 2.4 GHz spectrum congestion, motivating the range/interference characterization described in Phase 3 of Section 5 as a key input to selecting the dead-reckoning attenuation constant λ in Eq. (21).

6.2 Tracking Error Convergence

For smooth leader trajectories with persistent curvature (satisfying the persistency-of-excitation condition noted in Section 3.5), the follower's body-frame tracking error $\mathbf{e}[k]$ is expected to converge toward a small neighborhood of the origin, with residual steady-state error dominated by three practical, non-idealized effects rather than by the nominal control law itself: encoder quantization noise (bounding the resolution of the velocity estimate feeding the inner PID loop), differential wheel-diameter/friction mismatch between the two agents (a systematic bias only partially compensated by the calibration of Phase 2), and the finite bandwidth of the leader-state broadcast relative to a rapidly maneuvering leader. The adaptive gain scheduling of Section 4.3 is expected to reduce the transient settling time following a step change in tracking error — such as immediately after a formation-reconfiguration command or a dead-reckoning recovery — relative to a fixed-gain baseline, at the cost of a modest, bounded increase in steady-state chatter due to the transiently increased proportional gain; this trade-off is directly quantifiable by comparing the integral of absolute error (IAE) and settling time metrics, logged as described in Phase 5, with scheduling enabled versus disabled.

6.3 Resilience to Communication Dropout

With dead reckoning disabled, an induced communication dropout is expected to produce a tracking-error excursion that grows roughly in proportion to the leader's velocity and the dropout duration, since the follower's reference pose remains frozen while the leader continues to move. With dead reckoning enabled per Section 4.2, the same dropout is expected to produce a substantially smaller excursion, bounded primarily by the accuracy of the extrapolated velocity estimate over the dropout interval and by the exponential attenuation constant λ; excessively small λ (slow attenuation) risks compounding extrapolation error over long dropouts, while excessively large λ (fast attenuation) degrades the scheme toward the naive last-pose-hold baseline it is intended to improve upon, motivating λ as an empirically tuned parameter validated against the interference/range characterization of Phase 3.

6.4 Summary of Contributions

This report has presented a complete low-cost hardware and control architecture for two-agent leader–follower synchronization: an ESP8266-based compute and ESP-NOW-based communication platform requiring no supporting network infrastructure; a discrete-time unicycle kinematic and odometry model providing each agent with self-contained pose estimation; a cascaded

kinematic-plus-PID control architecture, together with a Routh–Hurwitz-grounded stability analysis of the inner velocity loop; and three enhancements — dynamic formation reconfiguration, packet-loss-resilient dead reckoning, and adaptive gain scheduling — that extend the baseline synchronization task toward the robustness and flexibility required of a genuinely field-deployable multi-agent system.

6.5 Scalability to N-Agent Swarms

The two-agent architecture generalizes to N agents along a path already anticipated by the design choices above. Because ESP-NOW's broadcast delivery mode was used for the leader-to-follower link rather than a unicast/acknowledged link, a single leader broadcast is received identically by any number of listening followers without additional leader-side overhead, so a single-leader, multiple-follower star topology extends immediately from the present two-agent case. Extending beyond a single-leader star topology to a general multi-hop or peer-to-peer swarm graph $\mathcal{G}$ would require each agent to both broadcast its own state and maintain a small table of neighbor states (rather than a single leader state), converting the present single-source dead-reckoning logic of Section 4.2 into a per-neighbor extrapolation, and converting the single-target tracking-error computation of Section 3.3 into a weighted combination of per-neighbor errors consistent with the general consensus update introduced in Section 1.1. The formation-offset mechanism of Section 4.1 already assigns an offset per follower *index*, so a formation of N agents follows directly by defining $N-1$ distinct offset vectors $\{\boldsymbol{\delta}_2, \ldots, \boldsymbol{\delta}_N\}$ relative to a single leader, without any change to the underlying control law. The principal open engineering questions for such an extension are (a) ESP-NOW's practical channel capacity and collision behavior as broadcast frequency and agent count both increase, and (b) whether a single-leader star topology remains adequate or whether a distributed, leaderless consensus law (of the form introduced in Eq. (1)) becomes necessary once the swarm must remain coordinated in the presence of leader agent failure — both natural directions for a follow-on capstone project building on the platform documented in this report.